ARCHITECTURES ORIENTEES SERVICES

Travaux Pratiques

5 cas pratiques Spring Boot

#	Cas Pratique	Technologie Clé	Durée
1	API REST Immobilier avec JWT	Spring Boot + JAX-RS + JWT	2h
2	Service SOAP Bancaire avec WSDL	Spring Boot + JAX-WS + WSDL	2h
3	Mashup Météo-Événements (appels parallèles)	Spring Boot + WebClient + CompletableFuture	2h30
4	API GraphQL Bibliothèque	Spring Boot + GraphQL	2h30
5	Architecture Event-Driven Livraison	Spring Boot + Apache Kafka	3h

CAS PRATIQUE 1

API REST Immobilier avec JWT

Technologie : Spring Boot 3 + Spring Security + JWT (stateless)

Objectif : Construire une API REST complète de gestion de biens immobiliers, sécurisée par JWT, avec les 6 contraintes de Fielding et le modèle de maturite Richardson niveau 2.

Durée estimée : 2 heures

Contexte et objectifs pédagogiques

Dans ce premier cas pratique, vous implémenterez une API REST qui respecte les contraintes de Fielding vues en cours : stateless (JWT), interface uniforme (verbes HTTP corrects, codes de statut sémantiques), cacheable (headers Cache-Control) et système en couches (Spring Security comme middleware).

Concepts couverts :

- Contrainte Stateless : authentification sans session côté serveur via JWT
- Interface Uniforme : CRUD complet avec GET / POST / PUT / PATCH / DELETE
- Codes HTTP sémantiques : 200, 201, 204, 400, 401, 403, 404
- Contrainte Cacheable : en-tetes Cache-Control et ETag
- Richardson Niveau 2 : ressources URI + verbes HTTP corrects

Structure du projet

```
src/
  main/
    java/com/tp/immobilier/
      config/
        SecurityConfig.java       # Configuration Spring Security + JWT
        JwtConfig.java            # Parametres JWT (secret, expiration)
      controller/
        AuthController.java       # POST /auth/login
        BiensController.java      # CRUD /api/v1/biens
      service/
        BiensService.java
        JwtService.java           # génération + validation du token
      model/
        Bien.java                 # @Entity JPA
        LoginRequest.java         # DTO
        BienRequest.java          # DTO creation/modification
      repository/
        BienRepository.java       # JpaRepository<Bien, Long>
      filter/
        JwtAuthFilter.java        # Filtre Spring Security
    resources/
      application.properties
  test/
    java/com/tp/immobilier/
```

```
        BiensControllerTest.java     # Tests avec MockMvc
```

Dépendances Maven (pom.xml)

```
<dependencies>
  <!-- Spring Boot Web (JAX-RS via Jersey ou Spring MVC) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Spring Security -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- JWT (jjwt) -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.3</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.3</version>
    <scope>runtime</scope>
  </dependency>

  <!-- JPA + H2 (base en memoire pour le TP) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Validation -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- Test -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Étapes de réalisation

Étape 1 — Modéliser l'entité Bien

```
@Entity
@Table(name = "biens")
public class Bien {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "L'adresse est obligatoire")
    @Column(nullable = false)
    private String adresse;

    @Positive(message = "Le prix doit etre positif")
    private BigDecimal prix;

    @DecimalMin("5.0")
    @DecimalMax("1000.0")
    private Double surfaceM2;

    @Min(1) @Max(20)
    private Integer nbPieces;

    @Enumerated(EnumType.STRING)
    private DpeEnum dpe; // enum A,B,C,D,E,F,G

    private Double latitude;
    private Double longitude;

    // getters / setters ...
}
```

Étape 2 — Créer le service JWT

```
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration-ms:3600000}")
    private long expirationMs; // 1h par defaut

    private SecretKey getKey() {
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }
```

```
    // Générer un token JWT signé
    public String generateToken(String username, List<String> roles) {
        return Jwts.builder()
            .subject(username)
            .claim("roles", roles)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getKey())
            .compact();
    }

    // Valider le token et extraire le username
    public String validateAndGetUsername(String token) {
        return Jwts.parser()
            .verifyWith(getKey())
            .build()
            .parseSignedClaims(token)
            .getPayload()
            .getSubject();
        // Lance JwtException si token invalide ou expire
    }
}
```

Étape 3 — Implémenter le controleur REST complet

```
@RestController
@RequestMapping("/api/v1/biens")
@Validated
public class BiensController {

    @Autowired private BiensService service;

    // GET /api/v1/biens?lat=48.8&lng=2.3&prixMax=500000
    @GetMapping
    public ResponseEntity<List<Bien>> rechercher(
            @RequestParam(required = false) Double lat,
            @RequestParam(required = false) Double lng,
            @RequestParam(required = false) BigDecimal prixMax) {

        List<Bien> biens = service.rechercher(lat, lng, prixMax);
        return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(5, TimeUnit.MINUTES))
            .eTag(String.valueOf(biens.hashCode())) // contrainte Cacheable
            .body(biens);
    }

    // GET /api/v1/biens/{id}
    @GetMapping("/{id}")
    public ResponseEntity<Bien> getById(@PathVariable Long id) {
        return service.findById(id)
            .map(bien -> ResponseEntity.ok()
                .cacheControl(CacheControl.maxAge(10, TimeUnit.MINUTES))
                .body(bien))
            .orElseThrow(() -> new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Bien " + id + " introuvable"));
```

```
    }

    // POST /api/v1/biens -> 201 Created
    @PostMapping
    public ResponseEntity<Bien> creer(
            @Valid @RequestBody BienRequest req,
            UriComponentsBuilder ucb) {

        Bien cree = service.creer(req);
        URI location = ucb.path("/api/v1/biens/{id}")
            .buildAndExpand(cree.getId()).toUri();
        return ResponseEntity.created(location).body(cree);
    }

    // PUT /api/v1/biens/{id} -> remplacement complet
    @PutMapping("/{id}")
    public ResponseEntity<Bien> modifier(
            @PathVariable Long id,
            @Valid @RequestBody BienRequest req) {
        return ResponseEntity.ok(service.modifier(id, req));
    }

    // PATCH /api/v1/biens/{id}/prix -> modification partielle
    @PatchMapping("/{id}/prix")
    public ResponseEntity<Void> modifierPrix(
            @PathVariable Long id,
            @RequestParam @Positive BigDecimal prix) {
        service.modifierPrix(id, prix);
        return ResponseEntity.noContent().build(); // 204
    }

    // DELETE /api/v1/biens/{id} -> 204 No Content
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> supprimer(@PathVariable Long id) {
        service.supprimer(id);
        return ResponseEntity.noContent().build();
    }
}
```

Tests a réaliser avec curl

1. Obtenir un token JWT :

```
curl -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"password"}'

# Réponse attendue :
# { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

2. Lister les biens (avec token) et vérifier les en-tetes de cache :

```
TOKEN="eyJhbGci...votre_token..."

curl -v http://localhost:8080/api/v1/biens \
  -H "Authorization: Bearer $TOKEN"
```

```
# Vérifier dans les en-tetes de réponse :
# Cache-Control: max-age=300
# ETag: "123456789"
```

3. Créer un bien et vérifier le 201 Created :

```
curl -X POST http://localhost:8080/api/v1/biens \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"adresse":"10 rue Victor Hugo","prix":350000,"nbPieces":3}'

# Vérifier : HTTP/1.1 201 Created
# Location: http://localhost:8080/api/v1/biens/1
```

TIP Pour aller plus loin : implémentez HATEOAS avec Spring HATEOAS (spring-boot-starter-hateoas). Ajoutez des liens _links.self et _links.modifier dans chaque réponse pour atteindre le niveau 3 de Richardson.

CAS PRATIQUE 2

Service SOAP Bancaire avec WSDL

Technologie : Spring Boot + Apache CXF + JAX-WS + WS-Security

Objectif : Développer un service SOAP de gestion de comptes bancaires exposant un WSDL, implémenter WS-Security pour l'authentification au niveau du message, et générer un client Java a partir du WSDL.

Durée estimée : 2 heures

Contexte et objectifs pédagogiques

Ce cas pratique illustre le cas d'usage ou SOAP est preferable a REST : les transactions financieres. Vous implémenterez un service SOAP avec Apache CXF (l'implementation JAX-WS la plus utilisée sous Spring Boot), générerez le WSDL automatiquement, et configurerez WS-Security pour une authentification au niveau du message XML (et non pas seulement au niveau du transport HTTPS).

Concepts couverts :

- Format Doc-Literal Wrapped (defaut JAX-WS)
- Structure WSDL : Types, Messages, PortType, Binding, Service
- SOAP Faults typees pour la gestion d'erreurs
- WS-Security : authentification par UsernameToken
- Génération automatique du client avec wsimport

Dépendances Maven (pom.xml)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Apache CXF : implementation JAX-WS pour Spring Boot -->
  <dependency>
    <groupId>org.apache.cxf</groupId>
    <artifactId>cxf-spring-boot-starter-jaxws</artifactId>
    <version>4.0.3</version>
  </dependency>

  <!-- WS-Security via WSS4J -->
  <dependency>
    <groupId>org.apache.cxf</groupId>
    <artifactId>cxf-rt-ws-security</artifactId>
    <version>4.0.3</version>
  </dependency>

  <!-- JPA + H2 -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

Étapes de réalisation

Étape 1 — Définir le contrat de service (SEI)

```
// Service Endpoint Interface (SEI) — le CONTRAT du service SOAP
@WebService(name = "ServiceBancaire",
            targetNamespace = "http://banque.tp.com/")
@SOAPBinding(style = Style.DOCUMENT,
             use = Use.LITERAL,
             parameterStyle = ParameterStyle.WRAPPED) // Doc-Literal Wrapped
public interface ServiceBancaireInterface {

    @WebMethod(operationName = "ObtenirSolde")
    @WebResult(name = "solde")
    BigDecimal obtenirSolde(
        @WebParam(name = "numeroCompte") String numeroCompte
    ) throws CompteIntrouveException;

    @WebMethod(operationName = "Virement")
    @WebResult(name = "referenceVirement")
    String effectuerVirement(
        @WebParam(name = "compteSource")      String compteSource,
        @WebParam(name = "compteDestination") String compteDestination,
        @WebParam(name = "montant")           BigDecimal montant,
        @WebParam(name = "motif")             String motif
    ) throws CompteIntrouveException, SoldeInsuffisantException;

    @WebMethod(operationName = "HistoriqueOperations")
    @WebResult(name = "operations")
    List<Operation> obtenirHistorique(
        @WebParam(name = "numeroCompte") String numeroCompte,
        @WebParam(name = "nbDerniers")   int nbDerniers
    ) throws CompteIntrouveException;
}
```

Étape 2 — Implémenter les SOAP Faults

```
// Exception metier mappee en SOAP Fault automatiquement par JAX-WS
@WebFault(name = "CompteIntrouveFault",
          targetNamespace = "http://banque.tp.com/")
public class CompteIntrouveException extends Exception {
```

```
    private final CompteIntrouveFault faultInfo;

    public CompteIntrouveException(String message, String numeroCompte) {
        super(message);
        this.faultInfo = new CompteIntrouveFault(message, numeroCompte);
    }

    public CompteIntrouveFault getFaultInfo() { return faultInfo; }
}

// Classe de details de la fault (sera serialisee dans <soap:Detail>)
@XmlRootElement
public class CompteIntrouveFault {
    private String message;
    private String numeroCompte;
    private LocalDateTime timestamp = LocalDateTime.now();
    // constructeur, getters...
}
```

Étape 3 — Configurer le service avec Apache CXF et WS-Security

```
@Configuration
public class CxfConfig {

    @Autowired
    private ServiceBancaireImpl serviceBancaire;

    // Expose le service sur /ws/banque
    @Bean
    public Endpoint endpointBancaire(Bus bus) {
        EndpointImpl endpoint = new EndpointImpl(bus, serviceBancaire);
        endpoint.publish("/banque");

        // Intercepteur entrant : valide le token WS-Security
        WSS4JInInterceptor inInterceptor = new WSS4JInInterceptor();
        Map<String, Object> inProps = new HashMap<>();
        inProps.put(WSHandlerConstants.ACTION,
            WSHandlerConstants.USERNAME_TOKEN);
        inProps.put(WSHandlerConstants.PASSWORD_TYPE,
            WSConstants.PW_TEXT);
        // Validateur : vérifie login/mot de passe
        inProps.put(WSHandlerConstants.PW_CALLBACK_CLASS,
            BanquePasswordCallback.class.getName());
        inInterceptor.setProperties(inProps);
        endpoint.getInInterceptors().add(inInterceptor);

        return endpoint;
    }
}

// application.properties
cxf.path=/ws
// WSDL accessible sur : http://localhost:8080/ws/banque?wsdl
```

Étape 4 — Tester avec SoapUI et générer un client Java

Tester depuis le terminal avec curl (message SOAP complet) :

```
curl -X POST http://localhost:8080/ws/banque \
  -H 'Content-Type: text/xml; charset=utf-8' \
  -d '
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:tns="http://banque.tp.com/">
  <soapenv:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/...">
      <wsse:UsernameToken>
        <wsse:Username>admin</wsse:Username>
        <wsse:Password>secret</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soapenv:Header>
  <soapenv:Body>
    <tns:ObtenirSolde>
      <tns:numeroCompte>FR76-0001-0002</tns:numeroCompte>
    </tns:ObtenirSolde>
  </soapenv:Body>
</soapenv:Envelope>'
```

Générer le client Java avec wsimport :

```
# Générer les classes client depuis le WSDL
wsimport -keep -verbose -d ./src/generated \
  http://localhost:8080/ws/banque?wsdl

# Classes générées automatiquement :
#   ServiceBancaire.java           <- factory du service
#   ServiceBancaireInterface.java  <- interface
#   ObtenirSolde.java              <- classe de requête
#   ObtenirSoldeResponse.java      <- classe de réponse
#   Operation.java                 <- modèle metier
```

ATTN SOAP retourne toujours HTTP 200, meme en cas d'erreur. L'erreur est dans le corps du message sous forme de <soap:Fault>. C'est une difference fondamentale avec REST qui utilisé les codes HTTP (404, 400, 500...).

CAS PRATIQUE 3

Mashup Météo-Événements (Appels Parallèles)

Technologie : Spring Boot + WebClient + CompletableFuture + Cache

Objectif : Réaliser un mashup qui combine une API météo, une API d'événements culturels et un service interne pour générer des recommandations personnalisees. Implémenter les appels parallèles pour optimiser la latence.

Durée estimée : 2 heures 30

Contexte et objectifs pédagogiques

Un mashup est une application qui combine plusieurs sources de données. Ce cas pratique illustre la composition de services SOA avec des appels parallèles (CompletableFuture), le cache HTTP pour respecter la contrainte Cacheable de Fielding, et la gestion résiliente des erreurs quand une API tierce est indisponible.

Concepts couverts :

- Composition de services SOA (mashup)
- Appels parallèles avec CompletableFuture.allOf() : temps total = max(latences)
- WebClient (reactive) pour les appels HTTP sortants
- Pattern Circuit Breaker pour la résilience
- Cache Redis pour reduire les appels aux APIs tierces

Architecture du mashup

```
                    +--------------------+
Client HTTP  ---> |  MashupController  |
GET /agenda       |  /api/agenda?ville |
                    +--------+-----------+
                             |
                    +--------v-----------+
                    |  AgendaService     | <- orchestre les appels
                    +---+-------+----+---+
                        |       |    |
        +-------------+  +----+    +----------+
        |                |                     |
+-------v-------+ +------v-------+ +----------v--------+
| MeteoService  | | EvenService  | | RecoService       |
| (API externe) | | (API extern) | | (service interne) |
+-------+-------+ +------+-------+ +-------------------+
        |                |
  openweather.org    ticketmaster.com

Tous les appels sont PARALLELES (CompletableFuture.allOf)
Temps total = max(tMeteo, tEvenements, tReco)  ~300ms
au lieu de  : tMétéo + tEvenements + tReco      ~900ms
```

Dépendances Maven

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
    <!-- WebClient est inclus dans webflux -->
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
  </dependency>

  <!-- Caffeine : cache en memoire simple pour le TP -->
  <dependency>
    <groupId>com.github.ben-manes.caffeine</groupId>
    <artifactId>caffeine</artifactId>
  </dependency>

  <!-- Résilience4j : circuit breaker -->
  <dependency>
    <groupId>io.github.résilience4j</groupId>
    <artifactId>résilience4j-spring-boot3</artifactId>
    <version>2.2.0</version>
  </dependency>
</dependencies>
```

Étapes de réalisation

Étape 1 — Configurer WebClient et le cache

```
@Configuration
@EnableCaching
public class WebClientConfig {

    @Bean
    public WebClient webClient() {
        return WebClient.builder()
            .codecs(c -> c.defaultCodecs().maxInMemorySize(1024 * 1024))
            .build();
    }

    @Bean
    public CaffeineCacheManager cacheManager() {
        CaffeineCacheManager manager = new CaffeineCacheManager();
        manager.setCaffeine(Caffeine.newBuilder()
            .expireAfterWrite(10, TimeUnit.MINUTES) // TTL cache 10min
            .maximumSize(500));
        return manager;
    }
}
```

Étape 2 Implémenter les services consommateurs

```
@Service
public class MeteoService {

    @Autowired private WebClient webClient;

    @Value("${météo.api.key}")
    private String apiKey;

    // Cache : evite de rappelér l'API pour la meme ville dans les 10 minutes
    @Cacheable(value = "météo", key = "#ville")
    @CircuitBreaker(name = "météoAPI", fallbackMethod = "metefallback")
    public CompletableFuture<MeteoDTO> getMeteo(String ville) {
        return webClient.get()
            .uri("https://api.openweathermap.org/data/2.5/weather"
                + "?q={ville}&appid={key}&units=metric&lang=fr",
                ville, apiKey)
            .retrieve()
            .onStatus(HttpStatusCode::is4xxClientError,
                r -> Mono.error(new VilleIntrouveException(ville)))
            .bodyToMono(MeteoDTO.class)
            .toFuture();
    }

    // Fallback si l'API météo est indisponible
    public CompletableFuture<MeteoDTO> metefallback(
            String ville, Throwable ex) {
        return CompletableFuture.complètedFuture(
            new MeteoDTO("Données météo indisponibles", ville));
    }
}
```

Étape 3 Orchestrer les appels PARALLELES dans AgendaService

```
@Service
public class AgendaService {

    @Autowired private MeteoService météoService;
    @Autowired private ÉvénementsService evenService;
    @Autowired private RecommandationService recoService;

    public AgendaResponse construireAgenda(String ville, String date) {

        long debut = System.currentTimeMillis();

        // LANCEMENT PARALLELE des 3 appels
        // Chaque appel retourne un CompletableFuture<T>
        CompletableFuture<MeteoDTO>          futureMétéo   =
            météoService.getMeteo(ville);
        CompletableFuture<List<ÉvénementDTO>> futureEven   =
            evenService.getEvenements(ville, date);
```

```
        CompletableFuture<List<RecoDTO>>      futureReco   =
            recoService.getRecommandations(ville);

        // Attente que TOUS les futures soient resolus
        // Temps total = MAX des 3 latences (pas leur somme !)
        CompletableFuture.allOf(futureMeteo, futureEven, futureReco)
            .join(); // bloquant mais dans un thread request Spring

        long tempsTotal = System.currentTimeMillis() - debut;
        // Log: "Agenda construit en Xms (parallèle)"

        return AgendaResponse.builder()
            .ville(ville)
            .date(date)
            .météo(futureMeteo.join())
            .événements(futureEven.join())
            .recommandations(futureReco.join())
            .tempsTraitementMs(tempsTotal)
            .build();
    }
}
```

Étape 4 — Comparer appels sequentiels vs parallèles

Ajoutez cet endpoint de comparaison dans votre controleur pour mesurer le gain reel :

```
// GET /api/agenda/benchmark?ville=Paris&date=2024-12-25
@GetMapping("/benchmark")
public ResponseEntity<BenchmarkResult> benchmark(
        @RequestParam String ville,
        @RequestParam String date) {

    // === SEQUENTIEL ===
    long t1 = System.currentTimeMillis();
    MeteoDTO météo            = météoService.getMeteo(ville).join();
    List<ÉvénementDTO> even   = evenService.getEvenements(ville,date).join();
    List<RecoDTO> reco        = recoService.getRecommandations(ville).join();
    long tempsSeq = System.currentTimeMillis() - t1;

    // === PARALLELE ===
    long t2 = System.currentTimeMillis();
    var fMétéo = météoService.getMeteo(ville);
    var fEven  = evenService.getEvenements(ville, date);
    var fReco  = recoService.getRecommandations(ville);
    CompletableFuture.allOf(fMeteo, fEven, fReco).join();
    long tempsPar = System.currentTimeMillis() - t2;

    return ResponseEntity.ok(BenchmarkResult.builder()
        .tempsSequentielMs(tempsSeq)    // ex: 900ms
        .tempsParallèleMs(tempsPar)     // ex: 310ms
        .gainFactor(tempsSeq / (double) tempsPar) // ex: 2.9x
        .build());
}
```

NOTE Pour simuler des latences realistes sans consommer de vrais credits API, implémentez des MockService avec Thread.sleep(200) dans la methode et annotez-les @Primary pour les tests. Cela permet de mesurer le gain de l'approche parallèle concrètement.

CAS PRATIQUE 4

API GraphQL Bibliothèque

Technologie : Spring Boot 3 + Spring for GraphQL

Objectif : Développer une API GraphQL complète pour une bibliothèque : livres, auteurs, emprunts. Illustrer la resolution du probleme d'over-fetching et d'under-fetching de REST, les mutations et les subscriptions temps reel.

Durée estimée : 2 heures 30

Contexte et objectifs pédagogiques

GraphQL repond aux limitations de REST : l'over-fetching (recevoir trop de champs) et l'under-fetching (faire plusieurs requêtes pour assembler les données). Dans ce TP, vous comparerez concrètement REST vs GraphQL sur le meme jeu de données, puis vous implémenterez Queries, Mutations et Subscriptions avec Spring for GraphQL.

Concepts couverts :

- Schéma Definition Language (SDL) GraphQL
- Queries : lecture de données avec selection de champs
- Mutations : creation, modification, suppression
- Subscriptions : notifications temps reel (WebSocket)
- DataLoader : eviter le probleme N+1 requêtes en base

Schéma GraphQL (src/main/resources/graphql/schéma.graphqls)

```
type Query {
    # Recherche de livres avec filtres optionnels
    livres(titre: String, genre: Genre, disponible: Boolean): [Livre!]!

    # Detail d'un livre avec ses relations
    livre(id: ID!): Livre

    # Liste des auteurs
    auteurs(nom: String): [Auteur!]!
}

type Mutation {
    # Emprunter un livre (retourne l'emprunt créé)
    emprunterLivre(livreId: ID!, adherentId: ID!): Emprunt!

    # Retourner un livre
    retournerLivre(empruntId: ID!): Emprunt!

    # Ajouter un nouveau livre (admin)
    ajouterLivre(input: LivreInput!): Livre!
}

type Subscription {
```

```
    # Notifier en temps reel quand un livre devient disponible
    livreDisponible(genre: Genre): Livre!
}

type Livre {
    id: ID!
    titre: String!
    isbn: String!
    genre: Genre!
    anneePublication: Int
    disponible: Boolean!
    auteur: Auteur!         # relation resolvee separement (DataLoader)
    empruntsActifs: [Emprunt!]!
}

type Auteur {
    id: ID!
    nom: String!
    nationalite: String
    livres: [Livre!]!       # relation inverse
}

type Emprunt {
    id: ID!
    dateEmprunt: String!
    dateRetourPrevue: String!
    dateRetourEffective: String
    livre: Livre!
    adherent: Adherent!
}

input LivreInput {
    titre: String!
    isbn: String!
    genre: Genre!
    auteurId: ID!
    anneePublication: Int
}

enum Genre {
    ROMAN SCIENCE_FICTION POLICIER BIOGRAPHIE HISTOIRE SCIENCE
}
```

Étapes de réalisation

Étape 1 — Configurer Spring for GraphQL

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
<dependency>
```

```
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
  <!-- necessaire pour les Subscriptions -->
</dependency>

# application.properties
spring.graphql.path=/graphql
spring.graphql.graphiql.enabled=true  # interface web de test
spring.graphql.graphiql.path=/graphiql
spring.graphql.websocket.path=/graphql-ws  # pour les Subscriptions
```

Étape 2 Implémenter les resolvers GraphQL

```
@Controller
public class LivreController {

    @Autowired private LivreService livreService;
    @Autowired private EmpruntService empruntService;

    // === QUERIES ===
    @QueryMapping
    public List<Livre> livres(
            @Argument String titre,
            @Argument Genre genre,
            @Argument Boolean disponible) {
        return livreService.rechercher(titre, genre, disponible);
    }

    @QueryMapping
    public Optional<Livre> livre(@Argument Long id) {
        return livreService.findById(id);
    }

    // === MUTATION ===
    @MutationMapping
    public Emprunt emprunterLivre(
            @Argument Long livreId,
            @Argument Long adherentId) {
        return empruntService.emprunter(livreId, adherentId);
    }

    @MutationMapping
    public Emprunt retournerLivre(@Argument Long empruntId) {
        Emprunt retour = empruntService.retourner(empruntId);
        // Notifier les subscribers que le livre est disponible
        livreDisponibleSink.tryEmitNext(retour.getLivre());
        return retour;
    }

    // === SUBSCRIPTION ===
    private final Sinks.Many<Livre> livreDisponibleSink =
        Sinks.many().multicast().onBackpressureBuffer();

    @SubscriptionMapping
```

```
    public Flux<Livre> livreDisponible(@Argument Genre genre) {
        return livreDisponibleSink.asFlux()
            .filter(l -> genre == null || l.getGenre() == genre);
    }
}
```

Étape 3 Implémenter le DataLoader (probleme N+1)

Sans DataLoader, charger 100 livres avec leurs auteurs = 1 requête livres + 100 requêtes auteurs = probleme N+1. Avec DataLoader, c'est 1 + 1 requêtes.

```
@Component
public class AuteurDataLoader implements
        BatchLoaderWithContext<Long, Auteur> {

    @Autowired private AuteurRepository auteurRepo;

    @Override
    public CompletionStage<List<Auteur>> load(
            List<Long> auteurIds,
            BatchLoaderEnvironment env) {

        // UNE SEULE requête SQL pour tous les auteurs demandes
        // au lieu de N requêtes individuelles
        List<Auteur> auteurs = auteurRepo.findAllById(auteurIds);

        // Retourner dans le meme ordre que les IDs demandes
        Map<Long, Auteur> map = auteurs.stream()
            .collect(Collectors.toMap(Auteur::getId, a -> a));
        return CompletableFuture.complètedFuture(
            auteurIds.stream().map(map::get).toList());
    }
}

// Declarer le DataLoader dans un @Bean :
@Bean
public RuntimeWiringConfigurer runtimeWiringConfigurer(
        AuteurDataLoader auteurLoader) {
    return builder -> builder.type("Livre", b ->
        b.dataFetcher("auteur", env -> {
            DataLoader<Long, Auteur> loader =
                env.getDataLoader("auteurs");
            Long auteurId = env.<Livre>getSource().getAuteurId();
            return loader.load(auteurId); // batch automatique
        }));
}
```

Étape 4 Tester avec GraphiQL et comparer avec REST

Ouvrez http://localhost:8080/graphiql et executez ces requêtes :

```
# Requête 1 : seulement titre + disponibilite (evite l'over-fetching)
query {
```

```
  livres(genre: ROMAN, disponible: true) {
    id
    titre
    disponible
  }
}

# Requête 2 : titre + auteur en une seule requête (evite l'under-fetching)
query {
  livres {
    titre
    auteur { nom nationalite }
    empruntsActifs { dateRetourPrevue adherent { nom } }
  }
}

# Mutation : emprunter un livre
mutation {
  emprunterLivre(livreId: "1", adherentId: "42") {
    id
    dateRetourPrevue
    livre { titre }
  }
}

# Subscription (depuis un onglet separé dans GraphiQL) :
subscription {
  livreDisponible(genre: ROMAN) {
    titre
    auteur { nom }
  }
}
# Puis effectuez un retour de livre pour voir la notification en temps reel
```

TIP Comparez la taille des réponses REST vs GraphQL : avec REST /api/livres, tous les champs sont retournes (~40 champs). Avec GraphQL en demandant seulement id + titre + disponible, la réponse est 10x plus petite. Mesurez la difference avec curl -w '%{size_download}'.

CAS PRATIQUE 5

Architecture Event-Driven Livraison

Technologie : Spring Boot + Apache Kafka + Spring Cloud Stream

Objectif : Implémenter une architecture event-driven pour un système de livraison e-commerce. Un service de commandes publié des événements, plusieurs services consommateurs (notification, stock, facturation) y reagissent independamment, illustrant le découplage maximal de la SOA.

Durée estimée : 3 heures

Contexte et objectifs pédagogiques

L'architecture event-driven pousse le principe de couplage faible de la SOA a son maximum : les services ne se connaissent pas. Le service de commandes publié un événement dans Kafka et ne sait pas qui y reagit. C'est le contraire d'une SOA synchrone ou le service appelant doit connaitre et appelér chaque service consomme.

Concepts couverts :

- Découplage total : fire-and-forget avec Kafka
- Topics Kafka et partitionnement
- Producteur (Producer) Spring : publication d'événements
- Consommateurs (Consumer Groups) : traitement independant
- Schéma d'événements avec Avro ou JSON Schéma
- Résilience : retry automatique, dead letter topic (DLT)

Architecture des microservices

```
+------------------+      +-----------+      +--------------------+
| Service Commandes|----> |   KAFKA   | ---> | Service Notif.     |
| :8081            |      |           |      | :8082 (email/SMS)  |
+------------------+      | Topics:   |      +--------------------+
                           | commandes |
   publié des événements   | .créée    | ---> +--------------------+
   sur chaque changement   | commandes |      | Service Stock      |
   de statut :             | .payee    |      | :8083              |
                           | commandes |      +--------------------+
   COMMANDE_CREEE          | .expediee |
   PAIEMENT_VALIDE         | commandes | ---> +--------------------+
   COMMANDE_EXPEDIEE       | .livree   |      | Service Facturation|
   COMMANDE_LIVREE         +-----------+      | :8084              |
   COMMANDE_ANNULEE                           +--------------------+

Chaque service peut tomber en panne independamment.
Kafka conserve les messages -> reprise automatique a la reconnexion.
```

Dépendances Maven

```
<!-- Dans chaque microservice (commandes, notification, stock, facturation) -->
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Spring Kafka -->
  <dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
  </dependency>

  <!-- JPA + H2 (chaque service a sa propre BD) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Test Kafka avec EmbeddedKafka -->
  <dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Étapes de réalisation

Étape 1 — Démarrer Kafka avec Docker Compose

```
# docker-compose.yml a la racine du projet
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on: [zookeeper]
    ports:
      - '9092:9092'
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
```

```
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    depends_on: [kafka]
    ports:
      - '8090:8080'
    environment:
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
    # Interface web : http://localhost:8090

# Démarrer : docker-compose up -d
# Vérifier : docker-compose ps
```

Étape 2 — Définir les événements metier

```
// Événement de base (tous les événements l'etendent)
public abstract class CommandeEvent {
    private final String commandeId;
    private final String type;
    private final Instant timestamp = Instant.now();
    // constructeur, getters...
}

// Événement : commande créée
public class CommandeCreeEvent extends CommandeEvent {
    private final String clientId;
    private final List<LigneCommande> lignes;
    private final BigDecimal montantTotal;
    private final String adresseLivraison;

    public CommandeCreeEvent(String commandeId, String clientId,
            List<LigneCommande> lignes, BigDecimal montant, String adresse) {
        super(commandeId, "COMMANDE_CREEE");
        this.clientId = clientId;
        this.lignes = lignes;
        this.montantTotal = montant;
        this.adresseLivraison = adresse;
    }
}

// Autres événements : PaiementValideEvent, CommandeExpedieeEvent,
// CommandeLivreeEvent, CommandeAnnuleeEvent
// -> meme structure, type different
```

Étape 3 — Implémenter le service Commandes (Producteur Kafka)

```
# application.properties du service-commandes
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.producer.key-
serializer=org.apache.kafka.common.serialization.StringSerializer
```

```
spring.kafka.producer.value-
serializer=org.springframework.kafka.support.serializer.JsonSerializer
spring.kafka.producer.properties.spring.json.add.type.headers=false
server.port=8081
```

```
@Service
public class CommandeService {

    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;
    @Autowired private CommandeRepository repo;

    private static final String TOPIC = "commandes";

    @Transactional
    public Commande créerCommande(CommandeRequest req) {
        // 1. Sauvegarder en base de données locale
        Commande cmd = new Commande();
        cmd.setId(UUID.randomUUID().toString());
        cmd.setStatut(StatutCommande.CREEE);
        cmd.setClientId(req.getClientId());
        cmd.setLignes(req.getLignes());
        cmd.setMontantTotal(req.getMontantTotal());
        Commande sauvegardee = repo.save(cmd);

        // 2. Publier l'événement dans Kafka (fire-and-forget)
        // Le service ne sait pas qui va traiter cet événement
        CommandeCreeEvent event = new CommandeCreeEvent(
            sauvegardee.getId(),
            req.getClientId(),
            req.getLignes(),
            req.getMontantTotal(),
            req.getAdresseLivraison()
        );

        // Clé = commandeId : garantit que tous les événements
        // d'une meme commande vont dans la meme partition
        kafkaTemplate.send(TOPIC, sauvegardee.getId(), event)
            .whenComplete((result, ex) -> {
                if (ex != null) {
                    log.error("Erreur publication Kafka", ex);
                }
            });

        return sauvegardee;
    }
}
```

Étape 4 — Implémenter les consommateurs (Notification, Stock, Facturation)

```
# application.properties du service-notification
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=service-notification  # ID unique du groupe
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-
deserializer=org.apache.kafka.common.serialization.StringDeserializer
```

```
spring.kafka.consumer.value-
deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.trusted.packages=com.tp.events
server.port=8082
```

```
@Service
public class NotificationConsumer {

    @Autowired private EmailService emailService;

    // Ecoute le topic 'commandes' pour les nouvelles commandes
    @KafkaListener(topics = "commandes",
                   groupId = "service-notification",
                   filter = "commandeCreeFilter")
    public void onCommandeCreee(CommandeCreeEvent event) {
        log.info("[NOTIF] Commande {} créée, envoi email...",
event.getCommandeId());
        emailService.envoyerConfirmation(
            event.getClientId(),
            event.getCommandeId(),
            event.getMontantTotal()
        );
    }

    @KafkaListener(topics = "commandes",
                   groupId = "service-notification",
                   filter = "commandeExpedieeFilter")
    public void onCommandeExpediee(CommandeExpedieeEvent event) {
        log.info("[NOTIF] Commande {} expediee, envoi tracking...",
            event.getCommandeId());
        emailService.envoyerTracking(
            event.getClientId(),
            event.getNumeroTracking()
        );
    }
}

// Service Stock : consommateur independant, meme topic
@Service
public class StockConsumer {

    @Autowired private StockRepository stockRepo;

    @KafkaListener(topics = "commandes", groupId = "service-stock")
    public void onÉvénementCommande(CommandeEvent event,
            @Header(KafkaHeaders.RECEIVED_KEY) String commandeId) {
        switch (event.getType()) {
            case "COMMANDE_CREEE" -> {
                // Reserver le stock pour chaque ligne de commande
                CommandeCreeEvent créée = (CommandeCreeEvent) event;
                créée.getLignes().forEach(l ->
                    stockRepo.réserver(l.getProduitId(), l.getQuantite()));
            }
            case "COMMANDE_ANNULEE" -> {
                // Liberer le stock reserve
                CommandeAnnuleeEvent annulee = (CommandeAnnuleeEvent) event;
                annulee.getLignes().forEach(l ->
```

```
                        stockRepo.libérer(l.getProduitId(), l.getQuantite()));
                }
            }
        }
    }
```

Étape 5 — Tester la résilience et vérifier le découplage

Scénario de test pour observer le découplage :

1. Démarrer tous les services (service-commandes, service-notification, service-stock)
2. Créer une commande via POST http://localhost:8081/api/commandes
3. Vérifier dans les logs que les 3 services reagissent independamment
4. STOPPER le service-notification (simuler une panne)
5. Créer plusieurs nouvelles commandes
6. REDEMARRER le service-notification
7. Observer que le service-notification traite AUTOMATIQUEMENT les commandes manquees (Kafka les avait conservees)

```
# Créer une commande (service-commandes doit etre demarre)
curl -X POST http://localhost:8081/api/commandes \
  -H 'Content-Type: application/json' \
  -d '{
    "clientId": "CLI-001",
    "adresseLivraison": "12 rue de la Paix, Paris",
    "lignes": [
      {"produitId": "PROD-A", "quantite": 2, "prixUnitaire": 29.99},
      {"produitId": "PROD-B", "quantite": 1, "prixUnitaire": 49.99}
    ]
  }'

# Vérifier les événements dans Kafka UI : http://localhost:8090
# -> Topic 'commandes' doit contenir l'événement

# Lister les messages dans un topic (en ligne de commande) :
docker exec -it kafka /bin/kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic commandes \
  --from-beginning
```

ATTN Chaque consommateur a son propre group-id. Si deux consommateurs partagent le meme group-id, Kafka considere qu'ils sont des replicas du meme service et ne livre le message qu'a un seul d'entre eux (load balancing). Utilisez des group-id distincts si chaque service doit traiter TOUS les messages.

TIP Pour aller plus loin : implémentez le pattern Outbox pour garantir que l'événement Kafka et la sauvegarde en BD sont atomiques. Sans Outbox, il est possible de sauvegarder en BD mais de rater la publication Kafka (ou vice versa) en cas de panne au mauvais moment.

Tableau de synthèse : correspondances cours / TP

Concept du cours	Cas pratique	Implementation concrète
Contrainte Stateless (Fielding)	Cas 1 - REST JWT	JwtService + JwtAuthFilter dans Spring Security
Contrainte Cacheable (Fielding)	Cas 1 - REST JWT	CacheControl.maxAge() + ETag dans ResponseEntity
Richardson Niveau 2	Cas 1 - REST JWT	GET/POST/PUT/PATCH/DELETE + codes HTTP sémantiques
SOAP Fault	Cas 2 - SOAP	@WebFault + exception mappee en <soap:Fault>
WSDL Doc-Literal Wrapped	Cas 2 - SOAP	@SOAPBinding(parameterStyle=WRAPPED)
WS-Security	Cas 2 - SOAP	WSS4JInInterceptor + BanquePasswordCallback
Mashup SOA	Cas 3 - Mashup	AgendaService orchestrant 3 services via WebClient
Appels parallèles	Cas 3 - Mashup	CompletableFuture.allOf() + .join()
Pattern Circuit Breaker	Cas 3 - Mashup	@CircuitBreaker(fallbackMethod=) de Résilience4j
Over/Under-fetching GraphQL	Cas 4 - GraphQL	Queries selectionnant exactement les champs voulus
Probleme N+1 requêtes	Cas 4 - GraphQL	BatchLoaderWithContext (DataLoader)
Subscriptions temps reel	Cas 4 - GraphQL	@SubscriptionMapping + Sinks.Many<T> + Flux<T>
Découplage SOA	Cas 5 - Kafka	KafkaTemplate.send() fire-and-forget
Résilience aux pannes	Cas 5 - Kafka	Consumer groups : reprise automatique apres panne
Composabilite services	Tous les cas	Services independants communiquant via interfaces

Bonne réalisation !